

Airbag ECU Portfolio

HS Harz — Functional Safety Engineering

AIRBAG ELECTRONIC CONTROL UNIT

Measurement and Processing of Sensor Signals to Control Airbag Operation

SOFTWARE ARCHITECTURE DESIGN SPECIFICATION AND IMPLEMENTATION (SADS)

Document Number	SADS-AIRBAG-ECU-001
Version	3.0
Date	2026-06-14
Status	RELEASED
ASIL Classification	ASIL D (ISO 26262) / SIL 3 (IEC 61508)
Target Hardware	Arduino Due (SAM3X8E ARM Cortex-M3, 84 MHz) ×2
Applicable Standards	ISO 26262:2018 / IEC 61508:2010
Author	Mariana Chavez Avila
Based On	HADS-AIRBAG-ECU-001 Rev 2.1 FSRs-AIRBAG-ECU-001 Rev 2.1 SRS-AIRBAG-SW-001 Rev 3.1
Firmware references	FINAL-CODE-AIRBAG-ECU-001

Table of Contents

1 Glossary	3
2 Overview	3
3 Tools and Programming Languages	4
3.1 Programming Language	4
3.2 Development Tools	4
4 Software Architecture Overview	4
4.1 Module Decomposition	4
4.2 Data Flow Between Modules	5
4.3 Execution Control Flow	5
5 System State Machine	6
5.1 State Definitions	6
5.2 Safe State Guarantees	6
6 Detailed Design of Software Modules	6
6.1 MOD-01 — Power-On Self-Test (POST)	6
6.1.1 CPU Test	6
6.1.2 RAM Test	7
6.1.3 ADC Test	7

6.1.4 POST Completion Criteria	7
6.2 MOD-02 — Sensor Acquisition	7
6.3 MOD-03 — Crash Detection	7
6.3.1 Scaling Formulas	7
6.3.2 Threshold Comparison	8
6.3.3 Threshold Rationale.....	8
6.4 MOD-04 — Deployment Control	8
6.4.1 Deployment Vote Logic.....	8
6.4.2 Event-triggered CAN TX (Rev 3.0 addition)	8
6.4.3 Deployment Authorization Truth Table	9
6.5 MOD-05 — CAN Communication.....	9
6.5.1 CAN Frame Layout (DLC=8 - Classical CAN Maximum)	9
6.5.2 Receive and Timeout.....	9
6.6 MOD-06 — Diagnostic Monitor	9
7 Occupant Classification and Baby Seat Logic	10
7.1 Signal Interpretation	10
7.2 Debounce Implementation (GAP-01 CLOSED Rev 3.0).....	10
7.3 Pull-up Configuration (HADS-CR-001 RESOLVED).....	10
8 Safety Mechanisms in Software.....	10
8.1 Hardware Watchdog Timer (WDT) — Pending	10
8.2 2oo2 Redundant Decision Architecture.....	11
8.3 Fail-Safe Direction Analysis	11
8.4 Permitted and Forbidden Software Operations.....	11
9 HW/SW Integration Procedure.....	11
9.1 Integration Steps	12
10 Compliance Gaps and Required Improvements	13
11 Verification Methods Overview	13
11.1 Applicable Verification Methods	14
11.2 Structural Coverage Target	14
12 Requirement Traceability Matrix	14
13 List of Dependent Documents.....	15
14 Approval and Sign-Off.....	16
15 Revision History	16

1 Glossary

Abbreviation	Definition
2oo2	Two-out-of-Two voting. Both ECUs must agree to trigger airbag deployment.
ADC	Analog-to-Digital Converter. Converts sensor voltages to 12-bit values (0–4095).
ASIL	Automotive Safety Integrity Level. Risk classification per ISO 26262 (QM, A, B, C, D).
ASIL D	Highest safety integrity level. Applied to the airbag deployment logic.
CAN	Controller Area Network. Communication bus between the two ECUs at 500 kbit/s.
CRC	Cyclic Redundancy Check. Error-detection code for CAN frames. Hardware CRC automatically generated and verified by SAM3X8E CAN controller.
DWI	Dash Warning Indicator. Fault warning lamp on pin D10. Must activate within ≤800 ms.
ECU	Electronic Control Unit. Each Arduino Due board acting as a redundant controller.
FSRS	Functional Safety Requirement. Traceable requirement from FSRS-AIRBAG-ECU-001.
GAP	Identified non-conformance between current firmware and an FSR requirement.
HADS	Hardware Architecture and Design Specification. Defines wiring, BOM, and pin assignments.
HARA	Hazard Analysis and Risk Assessment. Identifies hazards and derives ASIL classifications.
ITP	Implementation Test Plan. Defines unit tests, integration tests, and the code review checklist.
OCS	Occupant Classification System. Detects child seat presence via the baby seat switch (D11).
POST	Power-On Self-Test. Startup routine testing CPU, RAM, and ADC before the main loop.
SADS	Software Architecture and Design Specification. This document portfolio’s SW design reference.
SIL	Safety Integrity Level. Risk classification per IEC 61508 (SIL 1–4). ECU is SIL 3.
SPFM	Single-Point Fault Metric. ASIL D requires ≥99% of single-point faults to be covered.
TR	Test Report. Records test execution results, coverage evidence, and sign-off.
V&V	Verification and Validation. Confirms software is built correctly and meets safety goals.
WDT	Watchdog Timer. Resets the MCU if the main loop stalls. Required by FSR-22.
HW CRC	Hardware CRC by SAM3X8E CAN controller per ISO 11898 – 15-bit automatic CRC
voteDeploy	Deployment authorization computed in MOD-04

2 Overview

This Software Architecture and Design Specification (SADS) defines the internal software structure, module organisation, data flow, control flow, safety mechanisms, state machines, and implementation logic for the Airbag Electronic Control Unit (ECU) firmware running on two Arduino Due (SAM3X8E) microcontrollers in a redundant 2oo2 safety architecture.

The document covers the following architecture and design activities required by ISO 26262 for software at ASIL D:

- Software architecture design and tool/language selection
- Detailed module design and data structures
- HW/SW integration procedure
- Verification methods overview and requirement traceability
- Modification and change management procedure

Unit testing, integration testing, validation test cases, and test evidence are documented separately in the Implementation Test Plan (ITP-AIRBAG-ECU-001) and Test Report (TR-AIRBAG-ECU-001). The safety assessment is documented in the Functional Safety Assessment (FSA-AIRBAG-ECU-001).

This specification must be read in conjunction with FSRS-AIRBAG-ECU-001 Rev 2.1, SRS-AIRBAG-SW-001 Rev 3.1 and HADS-AIRBAG-ECU-001 Rev 2.1.

3 Tools and Programming Languages

Per ISO 26262-6:2018 Clause 5 and IEC 61508-3:2010 Clause 7.4, all tools and languages used in the development of safety-related software shall be qualified or otherwise justified. This section documents the selection rationale, qualification status, and constraints for each tool and language used.

3.1 Programming Language

Attribute	Detail	Safety Justification
Language	C/C++ (Arduino framework, Arduino IDE 2.3.6)	ISO 26262-6 Annex C supported language for embedded safety systems
Standard	C++11 (GCC 4.8.3 ARM bundled with Arduino Due board package)	Dynamic memory and recursion forbidden per ASIL D constraints
Forbidden constructs	malloc, new, delete, recursion, unbounded loops, delay() in loop()	Non-deterministic timing prohibited at ASIL D

3.2 Development Tools

Tool	Version	Role	Qualification / Justification
Arduino IDE	2.3.6	Code editor, compiler, upload tool	Tool output (compiled binary) verified by functional testing. No claim placed on tool correctness beyond standard compiler qualification.
GCC ARM (avr-g++)	4.8.3 (bundled)	Cross-compiler for SAM3X8E (ARM Cortex-M3)	Widely used in safety projects. Code output validated via module testing and integration tests documented in ITP-AIRBAG-ECU-001.
due_can library	v1.0 (open source)	Arduino Due CAN0 hardware abstraction	Limited to CAN frame send/receive. Qualified by code review + communication tests documented in ITP-AIRBAG-ECU-001.
USBCANV2 (OMS PC)	Latest stable	Passive CAN monitoring on OMS Lab PC	Used for test evidence collection only. Not part of deployed firmware. No safety claim placed on monitor tool.
Serial Monitor (Arduino IDE)	Bundled	Debug/diagnostic output at 115200 baud	Used for test observation only. No safety claim. Serial.print() calls are non-critical diagnostics.

4 Software Architecture Overview

The architecture is divided into six functional modules that map directly onto FSR requirements.

4.1 Module Decomposition

Module ID	Module Name	Primary FSR(s)	ASIL	Status Rev 3.0
MOD-01	Power-On Self-Test (POST)	FSR-17	ASIL C	Implemented — CPU/RAM/ADC tests

MOD-02	Sensor Acquisition	FSR-01, FSR-02	ASIL D	Implemented — 12-bit ADC, debounce added (GAP-01 CLOSED)
MOD-03	Crash Detection	FSR-01, FSR-02, FSR-03, FSR-04	ASIL D	Implemented — OR fusion of three sensors
MOD-04	Deployment Control	FSR-03, FSR-08, FSR-09, FSR-10	ASIL D	Implemented — voteDeploy + event-triggered TX
MOD-05	CAN Communication	FSR-13, FSR-14, FSR-15, FSR-16	ASIL B	Implemented — HW CRC (GAP-03 CLOSED), event TX added
MOD-06	Diagnostic Monitor	FSR-17, FSR-18, FSR-19, FSR-20, FSR-22	ASIL C/D	Implemented — CommFault, DWI, systemFault aggregation

4.2 Data Flow Between Modules

- MOD-02 reads rawA0, rawA1, rawA2 from 12-bit ADC and digital pin D11. Stores in global variables. Applies 10ms debounce to D11 (updateBabySeat()).
- MOD-03 reads rawA0–rawA2, scales to engineering units via toPhys(), applies threshold comparisons, writes localCrash boolean and float values accel, gyro, press.
- MOD-05 transmits local ECU status frame every 500ms and immediately on voteDeploy rising edge. Receives peer frame, writes remoteCrash, remoteFault, updates lastRemoteHB.
- MOD-06 evaluates fault flags (faultCPU, faultRAM, faultADC, faultCOMM), computes aggregated systemFault boolean.
- MOD-04 reads localCrash, remoteCrash, systemFault, remoteFault, babySeat. Computes voteDeploy. Drives D8, D9, D10.

4.3 Execution Control Flow

Step	Action	Period / Trigger	FSR
1	receiveCAN() — drain RX buffer, verify peer frame ID, update remoteCrash/remoteFault/lastRemoteHB	Every loop (non-blocking)	FSR-15
2	CAN timeout check — if millis()-lastRemoteHB > 2000ms → faultCOMM=TRUE	Every loop	FSR-15
3	updateBabySeat() — read D11, apply 10ms debounce filter	Every loop	FSR-11
4	analogRead A0, A1, A2 — raw 12-bit ADC values	Every loop	FSR-01, FSR-02
5	toPhys() — scale rawA0/A1/A2 to g, deg/s, kPa	Every loop	FSR-01
6	Threshold comparison — accelEx, gyroEx, pressEx → localCrash	Every loop	FSR-01, FSR-04
7	systemFault aggregation — OR of faultCPU, faultRAM, faultADC, faultCOMM	Every loop	FSR-20
8	voteDeploy logic — drive D8, D9, D10	Every loop	FSR-03, FSR-08
9	Event-triggered CAN TX — if voteDeploy rising edge → canSend() immediately	On rising edge	FSR-16
10	Periodic CAN TX — canSend() if 500ms elapsed	Every 500ms	FSR-16
11	Status LED blink — toggle D13 if 500ms elapsed	Every 500ms	—
12	Serial diagnostic print — if 1000ms elapsed	Every 1000ms	FSR-21

5 System State Machine

5.1 State Definitions

State ID	State Name	Conditions to Enter	Outputs	FSRS Ref
ST-01	POWER_ON	Reset/power-up	All outputs LOW, POST executing	SS-01
ST-02	POST_FAILED	Any POST returns FALSE	D10 HIGH, D8/D9 LOW — permanent safe inhibit	SS-05
ST-03	MONITORING	POST passed, no crash, no fault, peer CAN active	D10 LOW, D8/D9 LOW	SS-02
ST-04	FAULT_ACTIVE	systemFault=TRUE	D10 HIGH (DWI), D8/D9 LOW — deployment inhibit	SS-01, SS-04
ST-05	CRASH_DETECTED	localCrash AND remoteCrash AND !systemFault AND !remoteFault	D8 HIGH, D9 HIGH if !babySeat	SS-02

State transitions described in the graphical diagram produced in SADS Appendix (FLOW-CHART-AIRBAG-ECU-001).

5.2 Safe State Guarantees

- ST-02 (POST_FAILED): setup() never enters loop(). D8/D9 stay LOW. AND gate output permanently LOW. Safe inhibit SS-05.
- ST-04 (FAULT_ACTIVE): systemFault=TRUE forces voteDeploy=FALSE every cycle. D8/D9 LOW. D10 HIGH within one loop cycle (~1ms). FSR-19 (800ms) satisfied by large margin.
- CAN peer loss (faultCOMM=TRUE): After 2000ms without valid peer frame, faultCOMM propagates to systemFault, inhibiting deployment. Fail-safe direction: inhibit, not spurious deploy.
- Baby seat (babySeat=TRUE): D9 forced LOW unconditionally, even during valid crash. Passenger airbag suppressed. DWI stays OFF — active suppression is normal operation, not a fault.
-

6 Detailed Design of Software Modules

Each module is described with its interface, internal logic, data structures, and pseudo-code. Executed once in setup() before loop(). Results stored in sticky boolean fault flags. Any failed test transitions system to ST-02.

6.1 MOD-01 — Power-On Self-Test (POST)

Executed once in setup() before loop(). Results stored in sticky boolean fault flags. Any failed test transitions system to ST-02.

6.1.1 CPU Test

```
bool cpuTest() {
    volatile uint32_t a = 12345;
    volatile uint32_t b = 6789;
    volatile uint32_t c = a * b;
    return (c == 83810205UL); // volatile prevents compiler optimisation
}
```

Note: Rev 3.0 strengthened from addition test (a+b==300) to multiplication (12345*6789=83810205). More thorough ALU verification.

6.1.2 RAM Test

```
bool ramTest() {
    static volatile uint32_t mem[128];
    for (i=0;i<128;i++) mem[i]=0xAAAAAAAA;
    for (i=0;i<128;i++) { if(mem[i]!=0xAAAAAAAA) return false; }
    for (i=0;i<128;i++) mem[i]=0x55555555;
    for (i=0;i<128;i++) { if(mem[i]!=0x55555555) return false; }
    return true;
}
```

Rev 3.0: expanded from single-word to 128-word checkerboard sweep. Write 0xAAAAAAAA (alternating bits), verify, then write 0x55555555 (complement), verify. Satisfies FSR-22 RAM integrity at ASIL D.

6.1.3 ADC Test

```
bool adcTest() {
    int v = analogRead(A0);
    return (v >= 0 && v <= 4095);
}
```

6.1.4 POST Completion Criteria

Test	Pass Condition	Fail Action	Flag
CPU arithmetic	12345 * 6789 == 83810205	faultCPU=TRUE → systemFault → DWI ON, deploy inhibited	faultCPU
RAM checkerboard	128-word 0xAAAAAAAA / 0x55555555 verified	faultRAM=TRUE	faultRAM
ADC range	0 <= v <= 4095	faultADC=TRUE	faultADC
CAN init	Can0.begin() success	Not currently checked — GAP-05 open	—

6.2 MOD-02 — Sensor Acquisition

Reads all sensor inputs every loop iteration. Arduino Due analogReadResolution(12) set in setup().

```
uint16_t rawA0 = analogRead(A0); // Accelerometer POTI-1: 0-4095
uint16_t rawA1 = analogRead(A1); // Gyroscope POTI-2: 0-4095
uint16_t rawA2 = analogRead(A2); // Pressure POTI-3: 0-4095
updateBabySeat(); // D11 with 10ms debounce (GAP-01 CLOSED)
```

Sensor	Pin	ADC Range	Scaled Range	Deploy Threshold	ADC Threshold
Accelerometer POTI-1	A0	0-4095	0-100 g	>30.0 g	raw > 1228
Gyroscope POTI-2	A1	0-4095	0-360 deg/s	>120.0 deg/s	raw > 1365
Pressure POTI-3	A2	0-4095	0-500 kPa	>200.0 kPa	raw > 1638
Baby Seat Switch	D11	HIGH/LOW	TRUE/FALSE	LOW = baby seat	—

6.3 MOD-03 — Crash Detection

Converts raw ADC values to engineering units and applies threshold comparisons. An OR relationship between three sensors is used: any single sensor exceeding its threshold asserts localCrash = TRUE.

6.3.1 Scaling Formulas

```
float accel = (rawA0 / 4095.0) * 100.0; // 0-100 g
```

```
float gyro = (rawA1 / 4095.0) * 360.0; // 0-360 deg/s
float press = (rawA2 / 4095.0) * 500.0; // 0-500 kPa
```

6.3.2 Threshold Comparison

```
bool localCrash = (accel > 30.0) // > 30.0 g
                || (gyro > 120.0) // > 120.0 deg/s
                || (press > 200.0); // > 200.0 kPa
```

6.3.3 Threshold Rationale

Sensor	Threshold	Physical Meaning	Rationale
Accelerometer	30.0 g	Severe frontal impact pulse	Below hard braking (~1g), road bump (<5g); above required airbag crash pulse
Gyroscope	120.0 deg/s	Rollover onset angular rate	Above cornering (<30 deg/s); below confirmed rollover (>200 deg/s) — lab sensitivity
Pressure	200.0 kPa	~2 bar door panel deformation	Representative side-impact crush pressure exceeding ambient variation

6.4 MOD-04 — Deployment Control

Computes the deployment vote and drives all output pins. This is the safety-critical decision point combining all module inputs.

6.4.1 Deployment Vote Logic

```
// Step 1: aggregate system fault
systemFault = faultCPU || faultRAM || faultADC || faultCOMM;

// Step 2: compute deployment vote
bool voteDeploy = localCrash // this ECU detects crash
                  && remoteCrash // peer ECU independently detects crash
                  && !systemFault // no fault on this ECU
                  && !remoteFault; // no fault on peer ECU

// Step 3: drive output pins
digitalWrite(PIN_AIRBAG1, voteDeploy); // airbag 1 AND gate input
digitalWrite(PIN_AIRBAG2, voteDeploy && !babySeat); // airbag 2 - suppressed if baby seat
digitalWrite(PIN_WARNING, systemFault || remoteFault); // DWI on any fault
```

6.4.2 Event-triggered CAN TX (Rev 3.0 addition)

```
bool deployRisingEdge = (voteDeploy && !lastVoteDeploy);
lastVoteDeploy = voteDeploy;
if (deployRisingEdge || (millis() - lastTX > HEARTBEAT_MS)) {
    canSend(rawA0, rawA1, rawA2, localCrash, babySeat, systemFault);
    lastTX = millis();
}
```

Purpose: on first voteDeploy=TRUE, transmit CAN frame immediately rather than waiting up to 500ms for next heartbeat. Improves FSR-16 worst-case notification latency from 500ms to approximately 1 loop cycle (~1ms).

6.4.3 Deployment Authorization Truth Table

localCrash	remoteCrash	systemFault	remoteFault	babySeat	Airbag1 (D8)	Airbag2 (D9)
FALSE	any	any	any	any	LOW	LOW
TRUE	FALSE	any	any	any	LOW	LOW
TRUE	TRUE	TRUE	any	any	LOW	LOW
TRUE	TRUE	FALSE	TRUE	any	LOW	LOW
TRUE	TRUE	FALSE	FALSE	FALSE	HIGH	HIGH
TRUE	TRUE	FALSE	FALSE	TRUE	HIGH	LOW (suppressed)

6.5 MOD-05 — CAN Communication

6.5.1 CAN Frame Layout (DLC=8 - Classical CAN Maximum)

Byte	Field	Encoding	Notes
0	A0 low byte	rawA0 & 0xFF	Accelerometer ADC low byte
1	A0 high byte	rawA0 >> 8	Accelerometer ADC high byte
2	A1 low byte	rawA1 & 0xFF	Gyroscope ADC low byte
3	A1 high byte	rawA1 >> 8	Gyroscope ADC high byte
4	A2 low byte	rawA2 & 0xFF	Pressure ADC low byte
5	A2 high byte	rawA2 >> 8	Pressure ADC high byte
6	Status byte	Bit0=crash, Bit1=baby, Bit2=fault	Peer reads bits to extract remoteCrash and remoteFault
7	ECU ID	1 or 2	Identifies transmitting ECU

CRC integrity: The SAM3X8E CAN hardware controller automatically appends a 15-bit CRC to every transmitted frame and verifies it on every received frame per ISO 11898. No software CRC is required or implemented. GAP-03 CLOSED.

6.5.2 Receive and Timeout

```
void receiveCAN() {
    while (Can0.rx_avail()) {
        Can0.read(incoming);
        if (incoming.id != REMOTE_ID) continue; // filter peer ECU only
        lastRemoteHB = millis();
        remoteCrash = (incoming.data.bytes[6] & 0x01);
        remoteFault = (incoming.data.bytes[6] & 0x04);
    }
}

// Timeout: COMM_TIMEOUT = 2000ms (FSR-15 deviation — see GAP-08)
if (millis() - lastRemoteHB > COMM_TIMEOUT) faultCOMM = true;
else faultCOMM = false;
```

6.6 MOD-06 — Diagnostic Monitor

Runs every cycle. Aggregates hardware fault flags into systemFault. POST fault flags (faultCPU, faultRAM, faultADC) are sticky for the session; faultCOMM is evaluated dynamically.

Flag	Set When	Cleared When	Effect on System
------	----------	--------------	------------------

faultCPU	cpuTest() returns FALSE at POST	Never — sticky	systemFault=TRUE, DWI ON, deployment inhibited
faultRAM	ramTest() returns FALSE at POST	Never — sticky	systemFault=TRUE, DWI ON, deployment inhibited
faultADC	adcTest() returns FALSE at POST	Never — sticky	systemFault=TRUE, DWI ON, deployment inhibited
faultCOMM	millis() – lastRemoteHB > 2000 ms	Peer CAN frame received within 2000 ms	Dynamic: ST-04 while active, recovers to ST-03 when peer resumes

7 Occupant Classification and Baby Seat Logic

7.1 Signal Interpretation

D11 State	babySeat Value	Airbag 2 (D9)	Meaning
HIGH (~3.3 V via pull-up)	FALSE	Follows voteDeploy	No child seat — adult or empty — passenger airbag armed
LOW (switch pressed)	TRUE	Forced LOW	Child seat present — passenger airbag suppressed

Important: Active baby seat suppression (babySeat=TRUE) is normal fail-safe operation, not a fault condition. D10 (DWI) does NOT illuminate when baby seat suppression is active. DWI activates only on systemFault=TRUE.

7.2 Debounce Implementation (GAP-01 CLOSED Rev 3.0)

FSR-11 requires 10ms stable before state transition accepted. Fully implemented in Rev 3.2 firmware:

```
void updateBabySeat() {
    int current = digitalRead(PIN_BABY);
    if (current != lastRawBaby) {
        lastDebounce = millis();
        lastRawBaby = current;
    }
    if ((millis() - lastDebounce) >= DEBOUNCE_MS)
        stableBaby = current; // only accepted after 10ms stable
}
```

7.3 Pull-up Configuration (HADS-CR-001 RESOLVED)

Baby seat switch pull-up resistor (10kΩ) connected to Arduino Due 3.3V pin (not 5V). Previous connection to 5V rail would have over-voltaged the 3.3V SAM3X8E GPIO input. This was identified as HADS-CR-001 and resolved in hardware. D11 now reads 3.3V when button open (valid) and 0V when pressed (valid).

8 Safety Mechanisms in Software

8.1 Hardware Watchdog Timer (WDT) — Pending

FSR-22 requires hardware WDT. The SAM3X8E has an independent hardware WDT. Status: NOT YET ENABLED (HADS-CODE-OB-001). Required implementation before final V&V:

```
// In setup():
WDT->WDT_MR = WDT_MR_WDRSTEN | WDT_MR_WDV(256 * 10); // ~10ms timeout
// First line of loop():
WDT->WDT_CR = WDT_CR_KEY(0xA5) | WDT_CR_WDRSTT; // feed watchdog
```

8.2 2oo2 Redundant Decision Architecture

Both ECUs independently read identical sensor inputs, execute identical algorithms, compute independent deployment votes, and assert D8/D9 only when their local vote is TRUE. The DM74LS08N AND gate forms the hardware physical enforcement layer — LED fires only when BOTH boards simultaneously drive their respective input HIGH.

This architecture prevents: single-point hardware failures, single-channel software errors, single-sensor noise events, EMC spikes on one channel, and communication corruption.

8.3 Fail-Safe Direction Analysis

Failure Mode	Software Response	Hardware Response	Safe?
Peer CAN timeout (>2000ms)	faultCOMM=TRUE → systemFault → voteDeploy=FALSE → D8/D9 LOW	AND gate output LOW	Yes — inhibit on comm loss
POST failure (CPU/RAM/ADC)	faultXXX=TRUE → systemFault → deployment inhibited	D8/D9 never HIGH	Yes — inhibit on startup fault
Single sensor above threshold	localCrash=TRUE on this ECU only	AND gate requires BOTH D8 HIGH — peer must agree	Yes — no single-channel deploy
Single ECU software crash	Loop stops, D8/D9 go LOW (output default)	AND gate output LOW	Yes — fail LOW
Baby seat line open circuit	Pull-up holds D11 HIGH → babySeat=FALSE	D9 follows voteDeploy — not suppressed (GAP-02)	PARTIAL — GAP-02 open
voteDeploy=TRUE	Event-triggered CAN TX fires immediately	CAN frame reaches OMS PC within ~1ms	Improved FSR-16 coverage

8.4 Permitted and Forbidden Software Operations

Per ASIL D software development guidelines (ISO 26262-6 Annex A):

Permitted:

- Reading ADC pins A0, A1, A2 via analogRead()
- Reading digital pin D11 via digitalRead()
- Writing D8, D9 (deployment votes) and D10 (DWI) via digitalWrite()
- Transmitting CAN frames via Can0.sendFrame()
- Reading received CAN frames via Can0.read()
- Using millis() for timing comparisons
- Printing to Serial for diagnostics

Forbidden:

- Dynamic memory allocation (malloc, new) — non-deterministic timing prohibited at ASIL D
- Recursive functions — stack depth unpredictable
- Unbounded loops (while(true) without exit) — threatens deterministic cycle time
- Direct register access bypassing safety flags
- Clearing faultCPU, faultRAM, faultADC once set — these are sticky faults
- Modifying ECU_ID, DATA_ID, REMOTE_ID at runtime — compile-time constants only

9 HW/SW Integration Procedure

This section describes the procedure for integrating the firmware with the hardware assembly. It covers flash programming, pin verification, CAN bus bring-up, and the acceptance criteria that must be met before the integrated system is released to functional testing. Execution results and sign-off evidence are recorded in TR-AIRBAG-ECU-001.

9.1 Integration Steps

Step	Action	Acceptance Criterion	Owner
1	Assemble hardware per HADS-AIRBAG-ECU-001 Rev 2.0 Final — wiring, POTIs, baby seat switch (pull-up to 3.3V), CAN transceivers, AND gate, LEDs	All wiring verified against HADS Tables. No shorts. All signal lines continuous.	HW Engineer
2	Verify power rails: 5V to AND gate VCC (pin14), 3.3V to Waveshare VCC, 3.3V to baby seat pull-up, GND common	5V $\pm$ 0.25V at AND gate. 3.3V $\pm$ 0.15V at Waveshare. GND continuity.	HW Engineer
3	Flash ECU 1: ECU_ID=1, DATA_ID=0x100, REMOTE_ID=0x200	Upload complete. Serial: ECU 1 AIRBAG ECU Rev 3.2 STARTED. CPU/RAM/ADC: PASS.	SW Engineer
4	Flash ECU 2: ECU_ID=2, DATA_ID=0x200, REMOTE_ID=0x100	Upload complete. Serial: ECU 2 AIRBAG ECU Rev 3.2 STARTED. CPU/RAM/ADC: PASS.	SW Engineer
5	Verify CAN bus — connect OMS PC via USB-CAN-A, open USBCANv2 at 500 kbit/s	Frames ID 0x100 and 0x200 visible at ~500ms intervals. DLC=8. Byte 7 = 0x01 and 0x02.	HW/SW Engineer
6	Verify CommFault=0 on both ECUs via Serial Monitor	Serial: CommFault=0 on both. SystemFault=0. VoteDeploy=0.	SW Engineer
7	Verify rest state — all sensors below threshold	D8=LOW, D9=LOW, D10=LOW, D13 blinking. All LEDs OFF.	HW Engineer
8	Verify 2oo2 hardware AND gate — raise POTI-1 on ECU1 only above threshold	Airbag1 LED stays OFF. AND gate pin 3 LOW. ECU1: localCrash=1, voteDeploy=0.	HW Engineer
9	Verify full crash — raise POTI-1 on BOTH ECUs above threshold simultaneously	Airbag1 LED fires. AND gate pin 3 HIGH. Both ECUs: voteDeploy=1.	HW/SW Engineer
10	Verify baby seat suppression — press switch during step 9 crash	Airbag1 LED ON, Airbag2 LED OFF. D9 LOW on both ECUs.	HW/SW Engineer
11	Verify DWI — disconnect CAN between ECUs, wait >2100ms	D10 HIGH (DWI ON) on both ECUs. CommFault=1. VoteDeploy=0.	HW/SW Engineer
12	Sign off — all steps passed, record in TR-AIRBAG-ECU-001	Integration complete. Release to functional testing.	Project Lead

10 Compliance Gaps and Required Improvements

Status updated to reflect Rev 3.2 firmware baseline.

Gap ID	FSR	Description	Risk		Required Action
GAP-01	FSR-11	Baby seat debounce	Medium	CLOSED Rev 3.2	updateBabySeat() 10ms debounce implemented and tested
GAP-02	FSR-12	Baby seat line continuity monitoring — open-circuit stuck-HIGH not detected	High	OPEN	Add periodic D11 continuity test. Hardware limitation for lab scope.
GAP-03	FSR-13	Software CRC not in CAN frame	Medium	CLOSED — HW CRC	SAM3X8E hardware CAN controller provides ISO 11898 15-bit CRC automatically. No software CRC needed.
GAP-04	FSR-14	Parallel CAN bitstream readback not implemented	Low	OPEN — HW limit	Hardware limitation of Arduino Due classical CAN. Accepted for lab demo.
GAP-05	FSR-17	Can0.begin() success not checked in POST	Medium	OPEN	Check return value; set faultCOMM on failure.
GAP-06	FSR-22	Hardware WDT not enabled	High	OPEN — add before V&V	Enable WDT in setup(). Feed in loop(). HADS-CODE-OB-001.
GAP-07	FSR-21	Fault logging to EEPROM not implemented	Low	OPEN — lab scope	Serial output only. Accepted for lab demo scope.
GAP-08	FSR-15	CAN timeout 2000ms vs FSR-15 requirement 50ms	Medium	OPEN — formal deviation	50ms causes false faults with 500ms TX interval. NCR raised. 2000ms accepted with rationale.

11 Verification Methods Overview

This section defines the software verification approach required by ISO 26262-6:2018 at ASIL D. Detailed test cases, test execution records, pass/fail results, and coverage evidence are documented in ITP-AIRBAG-ECU-001 (test plan) and TR-AIRBAG-ECU-001 (test report).

11.1 Applicable Verification Methods

Method	Description	ISO 26262-6 Reference	Applicability
Code Review	Manual two-person review against SADS specs. Forbidden constructs, fault flags, threshold values, output logic, CAN encoding.	Clause 9.4.2	All modules — mandatory ASIL D
Static Analysis	Unreachable code, unused variables, ADC integer overflow, uninitialized variables, forbidden operations.	Clause 9.4.2	MOD-02, 03, 04
Unit Testing	Black-box and white-box per VVP-AIRBAG-ECU-001 v1.20.	Clause 9.4.3	MOD-01 through MOD-06
Integration Testing	HW/SW integration per Section 9 above and VVP v1.20.	Clause 9.4.3	Full system
Timing Analysis	Cycle time via micros(). CAN heartbeat timing via USBCANv2. Worst-case loop ≤1ms.	Clause 9.4.4	Main loop, CAN TX
Requirements Traceability	Each FSR traced to module and test case. RTM in VVP-AIRBAG-ECU-001 v1.20.	Clause 9.4.1	All FSR requirements

11.2 Structural Coverage Target

Coverage Metric	ASIL D Target	Applicable Modules
Statement Coverage	100%	All safety-relevant modules MOD-01 to MOD-06
Branch Coverage	100%	Crash threshold, baby seat suppression, safe-state transitions, POST pass/fail
MC/DC Coverage	100%	voteDeploy 4-condition AND, babySeat gate, crash OR fusion
Path Coverage	Best effort	Complex conditional paths in Zoo2 cross-comparison

12 Requirement Traceability Matrix

Forward traceability from every software requirement in SRS-AIRBAG-SW-001 Rev 3.1 to the software design element defined in this document:

SWR	Description	FSR	ASIL	Software Design Element
SWR-01	POST at boot	FSR-17	C	MOD-01 — cpuTest(), ramTest(), adcTest()
SWR-02	No output before POST	FSR-17	D	setup() — all outputs LOW before POST
SWR-03	ADC 12-bit	FSR-01	D	setup() — analogReadResolution(12)
SWR-04	Sensor read every loop	FSR-01,02	D	MOD-02 — analogRead A0/A1/A2

SWR-05	Physical conversion	FSR-01	D	MOD-03 — toPhys()
SWR-06	Threshold comparison	FSR-03,04	D	MOD-03 — accelEx/gyroEx/pressEx
SWR-07	Baby seat INPUT	FSR-09	D	setup() — pinMode(D11, INPUT) + 3.3V pull-up
SWR-08	10ms debounce	FSR-11	D	MOD-02 — updateBabySeat() (GAP-01 CLOSED)
SWR-09	Suppress-on-fault	FSR-10	D	MOD-02 — stableBaby retains last safe value
SWR-10	Sensor OR fusion	FSR-03	D	MOD-03 — localCrash = accelEx gyroEx pressEx
SWR-11	No blocking delay	FSR-02	D	Architecture — millis() timers throughout
SWR-12	D8 AND gate drive	FSR-03,08	D	MOD-04 — voteDeploy → D8
SWR-13	D9 pax suppression	FSR-09,10	D	MOD-04 — voteDeploy && !babySeat → D9
SWR-14	Fault inhibit	FSR-20	D	MOD-04/06 — systemFault → D8/D9 LOW
SWR-15	WDT config (pending)	FSR-22	D	MOD-01 — watchdog_setup() PENDING GAP-06
SWR-16	WDT feed (pending)	FSR-22	D	loop() first statement PENDING GAP-06
SWR-17	CAN TX 500ms + event	FSR-07,16	D	MOD-05 — canSend() heartbeat + rising edge
SWR-18	CAN timeout 2000ms	FSR-07,15	D	MOD-05/06 — COMM_TIMEOUT=2000ms (deviation)
SWR-19	CAN frame parsing	FSR-07	D	MOD-05 — receiveCAN() byte 6 extraction
SWR-20	CAN0 500kbps	FSR-13	B	MOD-05 — Can0.begin(CAN_BPS_500K)
SWR-21	CAN DLC=8	FSR-13,16	B	MOD-05 — canSend() frame layout
SWR-22	HW CRC ISO 11898	FSR-13	B	SAM3X8E CAN controller — automatic (GAP-03 CLOSED)
SWR-23	CAN 500ms + event	FSR-16	B	MOD-05 — HEARTBEAT_MS + deployRisingEdge
SWR-25	DWI remote fault	FSR-19	A	MOD-04 — remoteFault → D10 HIGH

13 List of Dependent Documents

Document ID	Title	Revision	Relationship
HARA-AIRBAG-ECU-001	Hazard Analysis and Risk Assessment	Rev 2.1	Source of hazard IDs and ASIL derivation
FSRS-AIRBAG-ECU-001	Functional Safety Requirements Specification	Rev 2.1	Primary source for FSR-01 to FSR-22
SC-AIRBAG-ECU-001	Safety Concept Document	Rev 2.0	Safety constraints SC-01 to SC-09

HADS-AIRBAG-ECU-001	Hardware Architecture and Design Specification	Rev 2.0	HW wiring, BOM, CAN IDs, pin assignments. CR-001 resolved.
SRS-AIRBAG-SW-001	Software Requirements Specification	Rev 3.1	SWR-01 to SWR-25. FSR compliance table. Code baseline
VVP-AIRBAG-ECU-001	Verification and Validation Plan	Rev 2.0	Test cases TC-VER-001 to TC-VAL-010. GAP-01/03 status updated.
FINAL-CODE-AIRBAG-ECU-001_2.ino	Firmware source code — final baseline	Rev 2.0	Event-triggered CAN TX. DLC=8. HW CRC. All tests passed.

14 Approval and Sign-Off

Role	Name	Signature	Date
Software Safety Engineer	Mariana Chavez Avila		
Hardware Safety Engineer	Atharva Avsare / Om Naik		
Functional Safety Engineer	Kruthika S. Gowda		
V&V Lead	Shivbaraje Bhosale		
Project Supervisor			

15 Revision History

Rev	Date	Author	Description
1.0	2026-05-28	Mariana Chavez Avila	Initial release (UART architecture). Full SADS covering architecture, module decomposition, state machine, safety mechanisms, and SWR traceability.
2.0	2026-06-12	Mariana Chavez Avila	Major revision – added sections, CAN architecture migration. UART inter-ECU link replaced by CAN bus (500 kbit/s). Master/Slave removed; peer-equal architecture. CAN frame byte layout documented. Compliance gap analysis (GAP-01–GAP-07). SRS traceability matrix.
3.0	2026-06-13	Mariana Chavez Avila	Firmware baseline updated to Rev 3.2. GAP-01 CLOSED (debounce implemented). GAP-03 CLOSED (hardware CRC confirmed). Event-triggered CAN TX added in MOD-04/05. canSend() replaces sendData(). DLC=8 confirmed. HADS-CR-001 resolved (pull-up 3.3V). All dependent document references updated. Execution flow updated with event TX step. Compliance gap table updated.